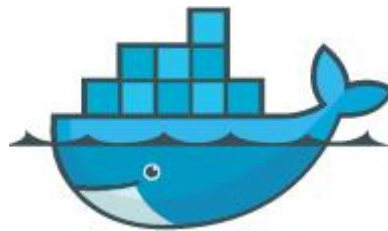


Lesson 1 Docker Introduction and Installation

Docker is an open-source application container engine based on the Go language and licensed under Apache License 2.0.

Docker enables developers to package their applications and dependencies into lightweight and portable containers, which can be deployed to popular Linux machines and virtualized environments.

Containers are fully isolated using sandboxing mechanisms, ensuring that there are no interfaces between them (similar to apps on an iPhone), and a more important aspect is that the containers have extremely low-performance overhead.



Docker Logo

Docker Official Website: <http://www.docker.com>

Docker Chinese Website: <https://www.docker-cn.com>

Docker Hub (Repository) Official Website: <https://hub.docker.com>

1. Docker Introduction

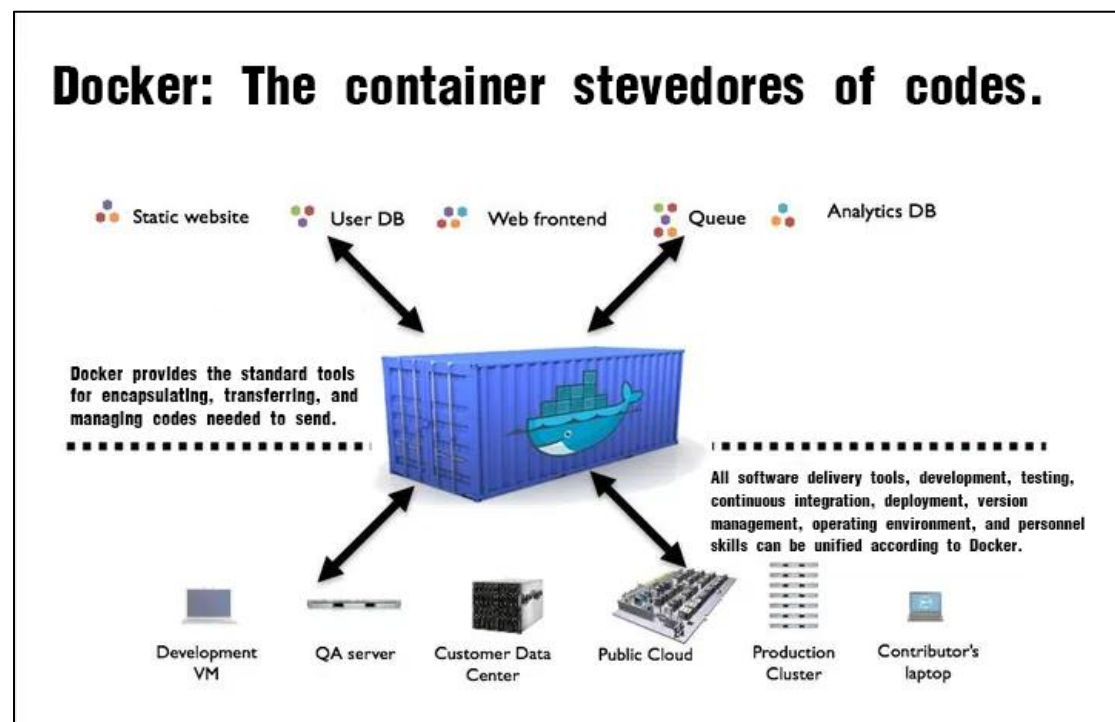
1.1 Docker Introduction

- Docker is a world-leading software container platform.
- Docker uses the Go language deployed by Google to achieve the development, based on Linux Kernel technologies such as “cgroup”, “namespace” , and “UnionFS (AUFS)” to encapsulate and isolate progress,

which belongs to virtualized technology at the operating system level. Because the processes of isolation are independent of the processes of the host and other isolation, Docker is also called a container. Its initial implementation is based on LXC.

- Docker automates repetitive tasks, such as the setup and configuration of the development environments, allowing developers to focus on the truly important thing: to build excellent software.

- You can create and use containers easily to run your applications. The containers can also manage, replicate, share, and modify versions, just like managing normal codes.



1.2 Docker Philosophy



The image shown above is the logo of Docker, which features a whale fully carrying containers on its back. The containers are isolated from each other,

representing the core concept of Docker.

For example, multiple applications run on the same server, which may cause conflicts in software port occupation. However, with Docker's isolation feature, each application can run independently. Additionally, Docker can maximize the utilization of server resources.

1.3 Docker Container Characteristics

- Lightweight

Multiple Docker containers running on a single machine can share the same operating system kernel, enabling them to quickly start and occupy minimal computing and memory resources. Images are constructed using file system layers and share some common files, which can greatly reduce disc usage and speed up image downloads.

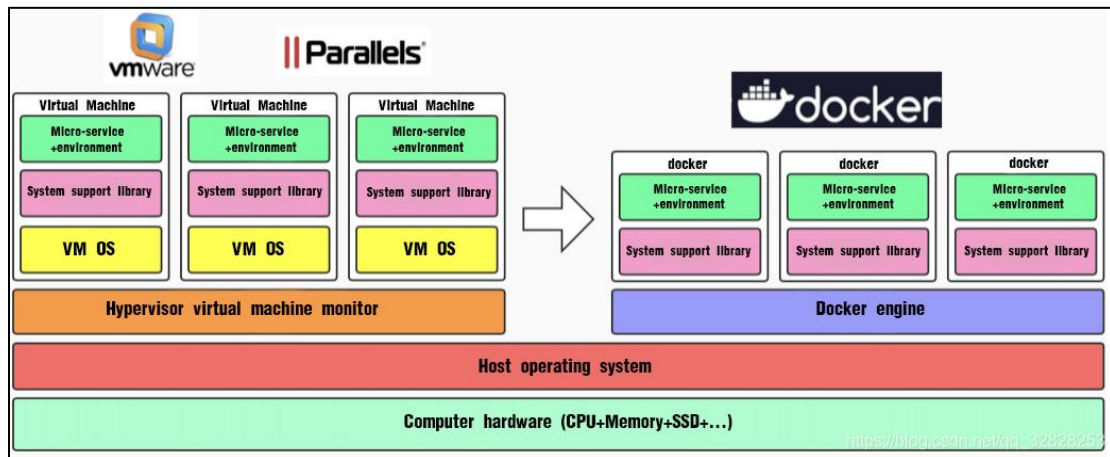
- Standard

Docker containers based on open standards can run on all mainstream Linux versions, Microsoft Windows, and any infrastructure, including VMs, bare-metal servers, and cloud platforms.

- Secure

Docker endows application isolation that is not only between applications but also independent of the underlying infrastructure. It provides the most powerful isolation by default. Therefore, if there is a problem in an application, it only affects a single container, rather than the entire machine.

1.4 Virtual Machines VS Docker



- The Docker daemon can directly communicate with the host operating system to distribute resources for each Docker container. It can also isolate the containers from the host operating system and each other. Virtual machines take several minutes to start, while the Docker containers can start within a few milliseconds. Since Docker has no overstuffed slave operating system, it can save a significant amount of disk space and other system resources.

- Virtual machines are more specialized in thoroughly isolating entire running environments. For example, cloud service providers usually adopt virtual machine technology to isolate different users, instead, Docker is widely used to isolate different applications such as front-end, back-end, and database.

- Docker containers are more resource-efficient and faster (in starting, closing, creating, and deleting) than virtual machines.

1.5 Docker Significance

- Consistent Running Environment

Docker images provide a complete runtime environment excluding the kernel, which ensures consistency in the application runtime environment to

avoid questions such as “This code works fine on my machine”.

- Faster Start Time

Docker can start within seconds or milliseconds, greatly saving the time of developing, testing, and deploying.

- Isolation

Docker’s isolation prevents resources from being affected by other users on shared servers.

- Elastic Scaling and Rapid Expansion

Docker is adept at dealing with concentrated usage pressure on servers.

- Transfer Convenience

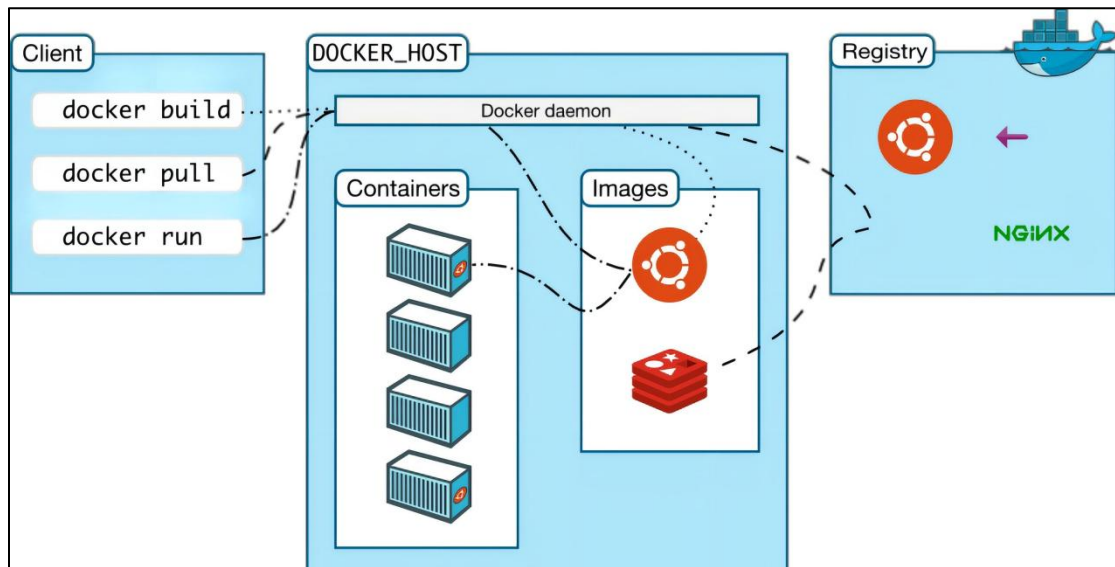
Docker can easily transfer applications running on one platform to another without worrying about the application malfunction caused by the changes in the runtime environment.

- Continuous Delivery and Deployment

Docker can use customized application images to realize continuous integration, delivery, and deployment.

1.6 Docker Architecture

Docker adopts a client-server architecture, where the Docker client communicates with its daemon responsible for building, operating, and distributing Docker containers. They can run on the same system, or the Docker client can be connected to a remote Docker daemon. They communicate with REST API over UNIX sockets or network interfaces. Another Docker client is Docker Compose, which enables you to manage applications composed of a set of containers.



1.7 Three Basic Concepts of Docker

1.7.1 Image --- A Special File System

An operating system consists of a kernel and a user space. As for Linux, after the kernel is started, it mounts a root file system to provide support for the user space. A Docker image is essentially a root file system.

A Docker image is a special file system, and it not only provides necessary files, such as programs, libraries, resources, and configurations, for container operation, but also includes some configuration parameters (such as anonymous volumes, environment variables, and users) to prepare for the runtime. The image does not contain any dynamic data, and its content remains unchanged after construction.

When designing Docker, the Union FS technology is fully utilized to design a layered storage architecture. The image is composed of multiple layers of file systems united together.

The Docker image is constructed layer by layer with the former layer serving as the basis for the latter layer. Once a layer is constructed, it remains unchanged, and any changes made in subsequent layers only affect that specific layer itself. For example, when a file from a previous layer is deleted in a later layer, the file is not deleted, but instead marked as deleted in the current

layer. When the container finally runs, the file is not visible, however, it is still present in the image. Therefore, when constructing a Docker image, please operate with caution, each layer should only include the necessary files and configurations. Any unnecessary files should be cleared before the layer construction is completed.

The layered storage design makes it easier to reuse and customize images. You can use a previously constructed image as a base layer to add new layers to customize the content and construct a new image according to your specific needs.

1.7.2 Container --- Entity of Image Runtime

The relationship between images and containers is similar to that between classes and instances in object-oriented programming. An image is a static definition, and a container is the entity that runs the image and can be created, started, stopped, deleted, paused, and more.

The essence of a container is a process, but it is different from a process executed directly on the host. The container process runs in its independent namespace. As mentioned earlier, images and containers both use hierarchical storage.

The lifecycle of a container storage layer is the same as that of the container itself. When a container is deleted, its storage layer is also deleted, resulting in the loss of any information stored in the storage layer.

According to the best practice requirement of Docker, containers should not write any data to their storage layer which should remain stateless. All file-writing operations should use data volumes or assign a host directory. This approach skips the storage layer to directly read and write to the host (or network storage), resulting in higher performance and stability. Data volumes have an independent lifecycle from containers. The data will not be lost even if the container is deleted. Therefore, using data volumes allows containers to be deleted and re-run without losing any data.

1.7.3 Repository --- The Place for Centralized Storage of Image File

Once an image is constructed, it can easily run on the current host. However, if the image needs to be used on other servers, a centralized storage and image distribution is required, such as Docker Registry.

A Docker Registry can contain multiple repositories. Each repository contains multiple tags and each tag corresponds to an image. Therefore, an Image Repository is the place for Docker to centrally store image files, which is similar to the commonly used code repository.

Generally, a repository includes images of different versions of the same software, and tags are usually used to correspond to each version. The “<Repository Name>: <Tag>” format can be used to specify which version of the software’s image to use. If a tag is not provided, the “latest” will be used as the default tag.

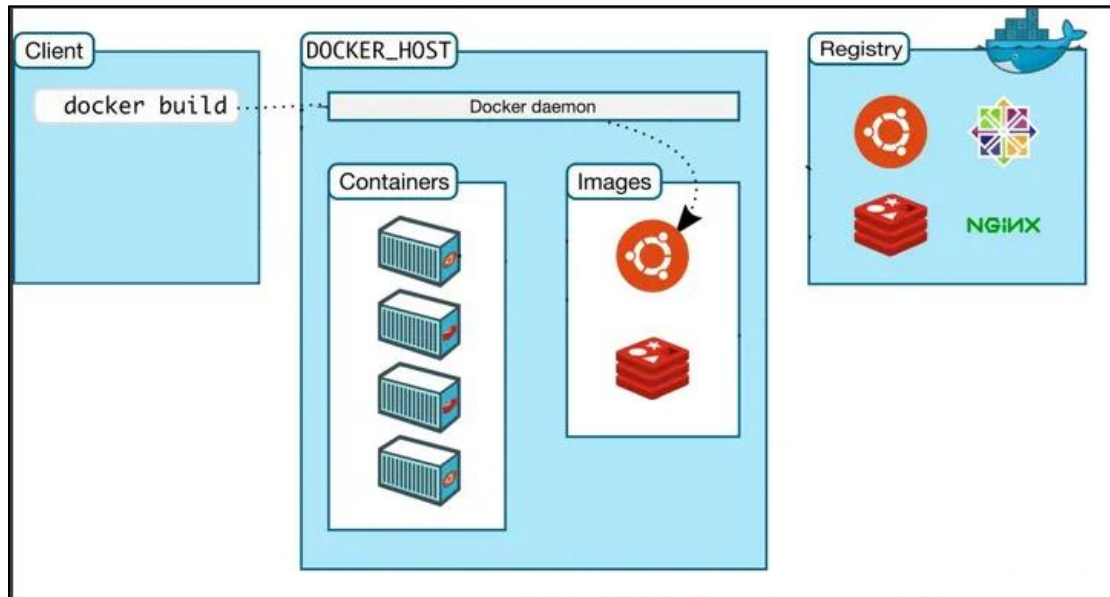
1.8 Docker Operating Mechanism

Docker utilizes the common CS (Client-Server) architecture, where the Docker client is responsible for processing various commands input by users. Such as “docker build” and “docker run”. The real executor is the Docker daemon which acts as the server. It’s worth noting that the Docker client can run on the same machine as the Docker daemon.

Next, let’s use a few commands to explain Docker’s workflow:

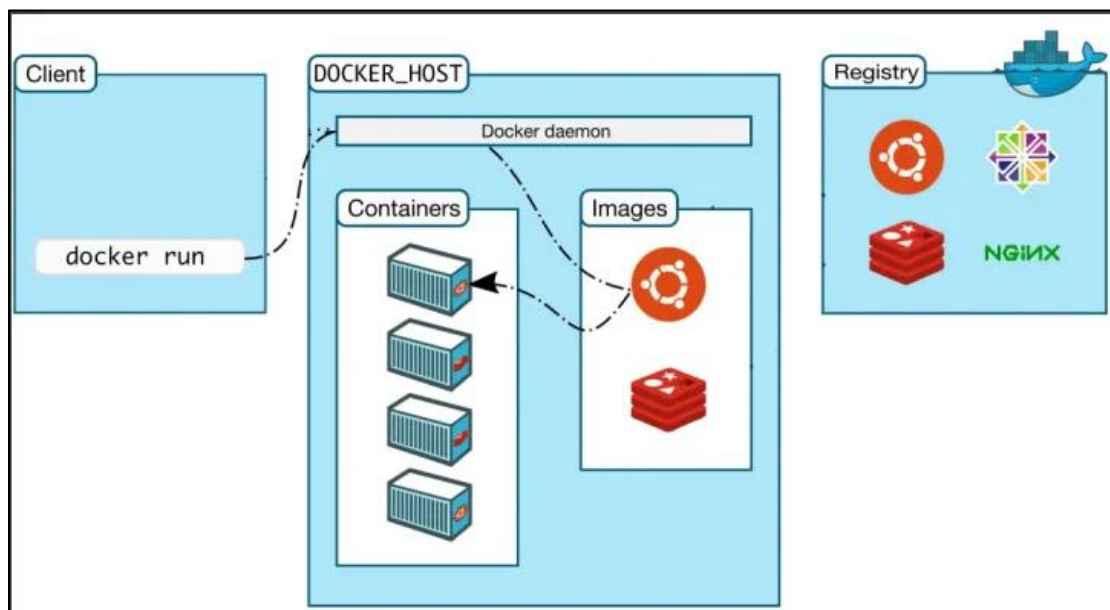
- docker build

This command is used when the “dockerfile” is written and handed over to Docker for "compilation". The client forwards the request to the Docker daemon, which then creates an “executable program” image based on the “dockerfile”.



- docker run

The “executable program” image can be run after obtained with the "docker run" command. The Docker daemon receives the command and locates the specific image, which is then loaded into memory and executed. The running instance of the image is called a container.



- docker pull

Users send a command using the Docker client, and the Docker daemon receives the command to send a request to the Docker Registry to download the image. Once downloaded, the image is stored locally and can be used.

2. Docker Installation

Official Installation Reference Manual: [Install Docker Engine on Debian | Docker Docs](#)

Note: Raspberry Pi's image already has installed Docker when you receive the package. The following steps are for learning and reference only.

Let's use the Aliyun source for downloading.

1) Press "Ctrl+Alt+T" to open the command line terminal, type "sudo apt-get update", and press "Enter" to update the apt package list.

```
pi@raspberrypi:~ $ sudo apt-get update
Hit:1 https://mirrors.aliyun.com/docker-ce/linux/debian bookworm InRelease
Hit:2 http://archive.raspberrypi.com/debian bookworm InRelease
Hit:3 http://deb.debian.org/debian bookworm InRelease
Hit:4 http://deb.debian.org/debian-security bookworm-security InRelease
Hit:5 http://deb.debian.org/debian bookworm-updates InRelease
Reading package lists... Done
```

2) Type the command "sudo apt-get install ca-certificates curl gnupg" to install the necessary certificates, tools, and GPG (GnuPG) key management tool.

```
pi@raspberrypi:~ $ sudo apt-get install ca-certificates curl gnupg
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20230311).
curl is already the newest version (7.88.1-10+deb12u5).
gnupg is already the newest version (2.2.40-1.1).
gnupg set to manually installed.
The following packages were automatically installed and are no longer required:
  libslirp0 slirp4netns
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 101 not upgraded.
```

3) Enter the command “sudo install -m 0755 -d /etc/apt/keyrings” to create a directory for storing the GPG keyring.

```
pi@raspberrypi:~ $ sudo install -m 0755 -d /etc/apt/keyrings
```

4) Type the command “curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg” to obtain the GPG key from Docker official and save it to the keyring.

```
pi@raspberrypi:~ $ curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
File '/etc/apt/keyrings/docker.gpg' exists. Overwrite? (y/N) y
```

```
pi@raspberrypi:~ $ sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

5) Enter the command “echo \"deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian \$(. /etc/os-release && echo \"\$VERSION_CODENAME\") stable\" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null” to add Docker storage library into Apt software package source list.

```
pi@raspberrypi:~ $ echo \"
deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian \
$(. /etc/os-release && echo \"$VERSION_CODENAME\") stable\" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

6) Enter “sudo apt-get update” to update the apt software package list.

```
pi@raspberrypi:~ $ sudo apt-get update
Get:1 https://download.docker.com/linux/debian bookworm InRelease [43.3 kB]
Hit:2 http://archive.raspberrypi.com/debian bookworm InRelease
Get:3 https://download.docker.com/linux/debian bookworm/stable arm64 Packages [14.2 kB]
Hit:4 http://deb.debian.org/debian bookworm InRelease
Hit:5 http://deb.debian.org/debian-security bookworm-security InRelease
Hit:6 http://deb.debian.org/debian bookworm-updates InRelease
Fetched 57.5 kB in 22s (2,597 B/s)
Reading package lists... Done
```

7) Enter “sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin” to install Docker and related components.

```
pi@raspberrypi:~ $ sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras docker-compose-plugin
Suggested packages:
```

8) Enter “sudo groupadd docker” to create a Docker user group.

```
pi@raspberrypi:~ $ sudo groupadd docker
```

9) Enter “sudo gpasswd -a \$USER docker” to add the current user to the Docker user group, which avoids having to enter “sudo” every time you run a Docker command.

```
pi@raspberrypi:~ $ sudo gpasswd -a $USER docker
Adding user pi to group docker
```

10) Enter “sudo reboot” to reboot Raspberry Pi.

```
pi@raspberrypi:~ $ sudo reboot
```

11) After the download is complete, enter “docker version” to check the Docker version.

```
pi@raspberrypi:~ $ docker version
Client: Docker Engine - Community
Version: 25.0.0
API version: 1.44
Go version: go1.21.6
Git commit: e758fe5
Built: Thu Jan 18 17:10:12 2024
OS/Arch: linux/arm64
Context: default

Server: Docker Engine - Community
Engine:
Version: 25.0.0
API version: 1.44 (minimum version 1.24)
Go version: go1.21.6
Git commit: 615dfdf
Built: Thu Jan 18 17:10:12 2024
OS/Arch: linux/arm64
Experimental: false
containerd:
Version: 1.6.27
GitCommit: a1496014c916f9e62104b33d1bb5bd03b0858e59
runc:
Version: 1.1.11
GitCommit: v1.1.11-0-g4bccb38
docker-init:
Version: 0.19.0
GitCommit: de40ad0
```

12) Enter “docker run hello-world”, if the message shown below appears, it means that the Docker is installed successfully.

```
pi@raspberrypi:~ $ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```